

Tests unitaires et fonctionnels

Guide à l'attention d'Arthur Berthault — MedIT

Vous nous avez demandé si les tests unitaires et fonctionnels pourraient vous aider à réduire les bugs sur vos projets web. Ce document répond à cette question simplement, avec des exemples concrets tirés de vos projets.

1. Qu'est-ce qu'un test unitaire ?

Le mot « unitaire » vient de « unité » : on teste **une seule et unique portion de code à la fois**, généralement une fonction.

Concrètement, un test unitaire pose la question : « Si je donne telle information à cette fonction, est-ce qu'elle me rend bien le résultat que j'attends ? »

Fonction : Bloc de code auquel on donne un nom, et qu'on peut appeler à tout moment pour effectuer une tâche précise. Exemple : une fonction « calculerTVA » qui prend un prix et retourne le montant avec la taxe.

Exemple concret pour MedIT : vous avez une fonction PHP qui calcule le prix d'une prestation avec remise. Un test unitaire vérifie que si on lui donne 100 € avec une remise de 20 %, elle retourne bien 80 €. Si demain quelqu'un modifie cette fonction et casse le calcul, le test l'alerte immédiatement.

2. Qu'est-ce qu'un test fonctionnel ?

Un test fonctionnel vérifie le comportement d'une **fonctionnalité complète du point de vue de l'utilisateur**. Plutôt que de tester un bout de code isolé, on simule le parcours d'un vrai visiteur sur le site.

Il ne s'intéresse pas à « comment le code fonctionne en interne », mais uniquement à « est-ce que le résultat visible est correct ? »

Scénario (terme utilisé en tests fonctionnels) : Suite d'actions enchaînées qui décrit le comportement d'un utilisateur. Exemple : « L'utilisateur ouvre la popup, remplit le formulaire, clique sur Envoyer, et voit un message de confirmation ».

Exemple concret pour MedIT : un test fonctionnel sur votre popup de salon vérifie que la popup s'ouvre bien au chargement de la page, que le formulaire est visible, qu'il peut être rempli et soumis, et qu'un message de confirmation apparaît. C'est exactement ce que contient votre cahier de recette.

3. Sur quels projets les utiliser ?

3.1 Les tests unitaires

Ils sont utiles uniquement lorsque le code contient une logique complexe (calculs, traitement de données, règles métier, c'est-à-dire les règles propres à l'activité de votre entreprise, qui sont traduites en code.

Exemples concrets pour MedIT :

- "Une remise ne peut pas dépasser 30 %"
- "Un devis expire après 30 jours"
- "Un accès à un dossier médical nécessite un double niveau d'authentification."

Ils sont donc adaptés à :

- Les applications développées entièrement sur mesure (PHP, JavaScript, etc.)
- Les modules WordPress sur mesure qui contiennent des fonctions métier (calcul de prix, traitement de formulaires, envoi d'e-mails automatiques)
- Les connexions à des systèmes externes (bases de données, logiciels médicaux)

En revanche, ils ne sont pas utiles sur un site vitrine sans logique complexe, ou sur un thème WordPress où l'on ne fait que du design.

3.2 Les tests fonctionnels

Ils s'appliquent à tous les projets avec une interface utilisateur, qu'ils soient réalisés sous WordPress ou sur mesure. Ils sont recommandés pour :

- Tout site avec des formulaires (contact, inscription, devis)
- Les espaces membres ou zones de connexion
- Les parcours d'achat ou de réservation
- La vérification avant livraison d'un site au client (c'est ce que fait votre cahier de recettes)

Bonne nouvelle : les tests fonctionnels ne nécessitent pas forcément d'être développeur. Des outils comme Cypress ou Playwright permettent d'automatiser ces vérifications, mais on peut aussi les réaliser manuellement, comme dans un cahier de recettes.

Cypress / Playwright : Outils informatiques qui permettent d'automatiser les tests fonctionnels : ils simulent les clics et la navigation d'un utilisateur sur un site, et vérifient que tout fonctionne correctement, sans intervention humaine.

4. Avantages et inconvénients

Critère	Tests unitaires	Tests fonctionnels
Rapidité	Très rapides à exécuter	Plus lents (simulent un navigateur)
Mise en place	Nécessite une architecture de code adaptée	Accessible, même sans grande expertise
Précision en cas de bug	Indique exactement quelle fonction est en cause	Indique qu'il y a un problème, pas où
Couverture	Très précise si bien écrits	Limitée aux scénarios prévus
Maintenance	À mettre à jour si le code est restructuré	Peut être fragilisé si l'interface change
Compréhensible par le client	Non (trop technique)	Oui (on voit ce qui fonctionne ou non)
Applicable à WordPress	Seulement sur le code sur mesure	Sur tous les sites WordPress

En résumé : les deux approches sont complémentaires. Pour vos projets MedIT, nous recommandons de combiner des tests unitaires sur les fonctions critiques (calculs, traitements de données médicales) et des tests fonctionnels sur les parcours utilisateur importants — en commençant par systématiser l'usage du cahier de recettes que vous avez déjà.